

```
In [1]: # =====
# PREDICTIVE HARDWARE FAILURE ANALYTICS
# STEP 1: COMPLETE DATA PREPROCESSING
# FINAL MEMORY-SAFE VERSION
# =====

# =====
# IMPORT LIBRARIES
# =====

import pandas as pd
import numpy as np
import os
import gc

from sklearn.preprocessing import StandardScaler
```

```
In [2]: # =====
# 1. LOAD DATASET
# =====

INPUT_FILE = r"C:\Users\admin\OneDrive\Desktop\DS- Activity 1\backblaze_analytic

OUTPUT_FOLDER = "output"

os.makedirs(
    OUTPUT_FOLDER,
    exist_ok=True
)

print("=" * 60)
print("DATASET LOADING")
print("=" * 60)

df = pd.read_csv(
    INPUT_FILE,
    low_memory=False
)

# Save original dataset information

original_rows = df.shape[0]

original_columns = df.shape[1]

print(
    "Dataset loaded successfully."
)

print(
    "Rows      :",
    f"{original_rows:,"}
)
```

```

print(
    "Columns :",
    original_columns
)

# =====
# MEMORY CHECK FUNCTION
# =====

def show_memory_usage(
    dataframe,
    message="Memory usage"
):

    memory_mb = (
        dataframe.memory_usage(
            deep=True
        ).sum()
        / (1024 ** 2)
    )

    print(
        f"{message}: "
        f"{memory_mb:.2f} MB"
    )

show_memory_usage(
    df,
    "Initial DataFrame memory"
)

```

```

=====
DATASET LOADING
=====
Dataset loaded successfully.
Rows    : 30,943,146
Columns : 19
Initial DataFrame memory: 9190.94 MB

```

```

In [3]: # =====
# 2. BASIC DATASET INSPECTION
# =====

print("\n" + "=" * 60)
print("FIRST 5 ROWS")
print("=" * 60)

print(
    df.head()
)

print("\n" + "=" * 60)
print("COLUMN NAMES")
print("=" * 60)

print(

```

```
df.columns.tolist()
)

print("\n" + "=" * 60)
print("DATA TYPES AND INFORMATION")
print("=" * 60)

df.info()
```

```
=====
FIRST 5 ROWS
=====
```

	date	serial_number	model	capacity_bytes	failure	\
0	2026-01-01	000a43e7dee60010	DELLBOSS VD	480036847616	0	
1	2026-01-01	00ac486b9bec0010	DELLBOSS VD	480036847616	0	
2	2026-01-01	01b776caeb6a0010	DELLBOSS VD	480036847616	0	
3	2026-01-01	02308a3415be0010	DELLBOSS VD	480036847616	0	
4	2026-01-01	0394cd7bc5d00010	DELLBOSS VD	480036847616	0	

	smart_5_normalized	smart_5_raw	smart_9_normalized	smart_9_raw	\
0	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	

	smart_90_normalized	smart_90_raw	smart_187_normalized	smart_187_raw	\
0	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	

	smart_188_normalized	smart_188_raw	smart_197_normalized	smart_197_raw	\
0	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	

	smart_198_normalized	smart_198_raw
0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

```
=====
COLUMN NAMES
=====
```

```
['date', 'serial_number', 'model', 'capacity_bytes', 'failure', 'smart_5_normalized', 'smart_5_raw', 'smart_9_normalized', 'smart_9_raw', 'smart_90_normalized', 'smart_90_raw', 'smart_187_normalized', 'smart_187_raw', 'smart_188_normalized', 'smart_188_raw', 'smart_197_normalized', 'smart_197_raw', 'smart_198_normalized', 'smart_198_raw']
```

```
=====
DATA TYPES AND INFORMATION
=====
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30943146 entries, 0 to 30943145
Data columns (total 19 columns):
#   Column              Dtype
---  -
0   date                 str
1   serial_number        str
2   model                str
3   capacity_bytes       int64
4   failure              int64
```

```

5 smart_5_normalized float64
6 smart_5_raw float64
7 smart_9_normalized float64
8 smart_9_raw float64
9 smart_90_normalized float64
10 smart_90_raw float64
11 smart_187_normalized float64
12 smart_187_raw float64
13 smart_188_normalized float64
14 smart_188_raw float64
15 smart_197_normalized float64
16 smart_197_raw float64
17 smart_198_normalized float64
18 smart_198_raw float64
dtypes: float64(14), int64(2), str(3)
memory usage: 4.4 GB

```

```

In [4]: # =====
# 3. CHECK MISSING VALUES
# =====

print("\n" + "=" * 60)
print("MISSING VALUES BEFORE PREPROCESSING")
print("=" * 60)

missing_values = df.isnull().sum()

print(
    missing_values
)

print(
    "\nMissing Value Percentage:"
)

missing_percentage = (
    df.isnull().sum()
    / len(df)
) * 100

print(
    missing_percentage.sort_values(
        ascending=False
    )
)

```

```
=====
MISSING VALUES BEFORE PREPROCESSING
=====
date                0
serial_number       0
model               0
capacity_bytes      0
failure             0
smart_5_normalized  208717
smart_5_raw         208717
smart_9_normalized  39856
smart_9_raw         39856
smart_90_normalized 26718632
smart_90_raw        26718632
smart_187_normalized 20675970
smart_187_raw       20675970
smart_188_normalized 20703124
smart_188_raw       20703124
smart_197_normalized 775206
smart_197_raw       775206
smart_198_normalized 237727
smart_198_raw       237727
dtype: int64
```

Missing Value Percentage:

```
smart_90_normalized  86.347497
smart_90_raw        86.347497
smart_188_normalized 66.906978
smart_188_raw       66.906978
smart_187_normalized 66.819224
smart_187_raw       66.819224
smart_197_raw       2.505259
smart_197_normalized 2.505259
smart_198_normalized 0.768270
smart_198_raw       0.768270
smart_5_normalized  0.674518
smart_5_raw         0.674518
smart_9_normalized  0.128804
smart_9_raw         0.128804
date                0.000000
model               0.000000
serial_number       0.000000
failure             0.000000
capacity_bytes      0.000000
dtype: float64
```

```
In [5]: # =====
# 4. IDENTIFY NUMERICAL AND CATEGORICAL COLUMNS
# =====

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()
```

```

print("\n" + "=" * 60)
print("NUMERICAL COLUMNS")
print("=" * 60)

print(
    numeric_columns
)

print("\n" + "=" * 60)
print("CATEGORICAL COLUMNS")
print("=" * 60)

print(
    categorical_columns
)

```

```

=====
NUMERICAL COLUMNS
=====
['capacity_bytes', 'failure', 'smart_5_normalized', 'smart_5_raw', 'smart_9_norma
lized', 'smart_9_raw', 'smart_90_normalized', 'smart_90_raw', 'smart_187_normaliz
ed', 'smart_187_raw', 'smart_188_normalized', 'smart_188_raw', 'smart_197_normali
zed', 'smart_197_raw', 'smart_198_normalized', 'smart_198_raw']

=====
CATEGORICAL COLUMNS
=====
['date', 'serial_number', 'model']

```

```

In [6]: # =====
# 5. HANDLE MISSING VALUES
# =====

print("\n" + "=" * 60)
print("HANDLING MISSING VALUES")
print("=" * 60)

# -----
# Numerical columns
# Fill missing values using median
# -----

for column in numeric_columns:

    if df[column].isnull().any():

        median_value = df[
            column
        ].median()

        df[column] = df[
            column
        ].fillna(
            median_value
        )

# -----

```

```

# Categorical columns
# Fill missing values using mode
# -----

for column in categorical_columns:

    if df[column].isnull().any():

        mode_values = df[
            column
        ].mode(
            dropna=True
        )

        if len(mode_values) > 0:

            df[column] = df[
                column
            ].fillna(
                mode_values.iloc[0]
            )

        else:

            df[column] = df[
                column
            ].fillna(
                "Unknown"
            )

print(
    "Total missing values after handling:",
    int(
        df.isnull().sum().sum()
    )
)

```

```

=====
HANDLING MISSING VALUES
=====
Total missing values after handling: 0

```

```

In [7]: # =====
# 6. CHECK DUPLICATES
# =====

print("\n" + "=" * 60)
print("DUPLICATE RECORDS")
print("=" * 60)

duplicates_before = (
    df.duplicated().sum()
)

print(
    "Duplicates found:",
    duplicates_before
)

```



```

)

if duplicates_before > 0:

    df = df.drop_duplicates(
        ignore_index=True
    )

duplicates_after = (
    df.duplicated().sum()
)

print(
    "Duplicates after removal:",
    duplicates_after
)

gc.collect()

```

```

=====
DUPLICATE RECORDS
=====
Duplicates found: 345662
Duplicates after removal: 0

```

Out[7]: 0

```

In [8]: # =====
# 7. IDENTIFY POSSIBLE TARGET COLUMN
# =====

print("\n" + "=" * 60)
print("TARGET COLUMN DETECTION")
print("=" * 60)

possible_targets = [
    "failure",
    "failed",
    "disk_failure",
    "disk_failed",
    "failure_status",
    "target",
    "label"
]

target_column = None

# -----
# Exact match
# -----

for column in possible_targets:

    if column in df.columns:

```

```

        target_column = column

        break

# -----
# Case-insensitive match
# -----

if target_column is None:

    lower_column_mapping = {
        column.lower(): column
        for column in df.columns
    }

    for target_name in possible_targets:

        if (
            target_name.lower()
            in lower_column_mapping
        ):

            target_column = (
                lower_column_mapping[
                    target_name.lower()
                ]
            )

            break

if target_column is not None:

    print(
        "Target column detected:",
        target_column
    )

else:

    print(
        "No automatic target column detected."
    )

```

```

=====
TARGET COLUMN DETECTION
=====
Target column detected: failure

```

```

In [9]: # =====
# 8. OUTLIER DETECTION USING IQR
# =====

print("\n" + "=" * 60)
print("OUTLIER DETECTION")
print("=" * 60)

```

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

outlier_columns = (
    numeric_columns.copy()
)

if (
    target_column is not None
    and target_column in outlier_columns
):

    outlier_columns.remove(
        target_column
    )

outlier_summary = {}

for column in outlier_columns:

    Q1 = df[
        column
    ].quantile(
        0.25
    )

    Q3 = df[
        column
    ].quantile(
        0.75
    )

    IQR = Q3 - Q1

    lower_bound = (
        Q1 - 1.5 * IQR
    )

    upper_bound = (
        Q3 + 1.5 * IQR
    )

    outlier_count = (
        (
            df[column] < lower_bound
        )
        |
        (
            df[column] > upper_bound
        )
    )
```

```

    )
).sum()

outlier_summary[
    column
] = int(
    outlier_count
)

print(
    f"{column}: "
    f"{outlier_count:,} outliers"
)

```

```

=====
OUTLIER DETECTION
=====
capacity_bytes: 1,868,933 outliers
smart_5_normalized: 136,230 outliers
smart_5_raw: 1,570,312 outliers
smart_9_normalized: 0 outliers
smart_9_raw: 4,462 outliers
smart_90_normalized: 0 outliers
smart_90_raw: 3,941,911 outliers
smart_187_normalized: 456,490 outliers
smart_187_raw: 457,208 outliers
smart_188_normalized: 796 outliers
smart_188_raw: 662,232 outliers
smart_197_normalized: 23,502 outliers
smart_197_raw: 612,655 outliers
smart_198_normalized: 21,168 outliers
smart_198_raw: 423,526 outliers

```

```

In [10]: # =====
# 9. OUTLIER HANDLING USING IQR CAPPING
# =====

print("\n" + "=" * 60)
print("HANDLING OUTLIERS")
print("=" * 60)

for column in outlier_columns:

    Q1 = df[
        column
    ].quantile(
        0.25
    )

    Q3 = df[
        column
    ].quantile(
        0.75
    )

```

```

IQR = Q3 - Q1

lower_bound = (
    Q1 - 1.5 * IQR
)

upper_bound = (
    Q3 + 1.5 * IQR
)

df[column] = df[
    column
].clip(
    lower=lower_bound,
    upper=upper_bound
)

print(
    "Outlier capping completed."
)

```

```

=====
HANDLING OUTLIERS
=====
Outlier capping completed.

```

```

In [11]: # =====
# 10. CATEGORICAL DATA ANALYSIS
# =====

print("\n" + "=" * 60)
print("CATEGORICAL DATA ANALYSIS")
print("=" * 60)

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()

if len(categorical_columns) == 0:
    print(
        "No categorical columns found."
    )
else:
    for column in categorical_columns:
        print(
            "\nColumn:",
            column
        )

```

```
print(  
    "Unique values:",  
    df[column].nunique(  
        dropna=False  
    )  
)  
  
print(  
    df[column]  
    .value_counts()  
    .head(10)  
)
```

```
=====
CATEGORICAL DATA ANALYSIS
=====
```

Column: date

Unique values: 90

date

2026-03-31	345662
2026-03-30	345604
2026-03-26	344751
2026-03-20	344636
2026-03-21	344565
2026-03-19	344551
2026-03-17	344470
2026-03-29	344468
2026-03-18	344456
2026-03-24	344345

Name: count, dtype: int64

Column: serial_number

Unique values: 351095

serial_number

000a43e7dee60010	90
00ac486b9bec0010	90
01b776caeb6a0010	90
02308a3415be0010	90
0394cd7bc5d00010	90
043d3be09f4a0010	90
04a81d26aebf0010	90
0559e3d009d20010	90
0564f6f3fab90010	90
059bba587f390010	90

Name: count, dtype: int64

Column: model

Unique values: 80

model

WDC WUH722222ALE6L4	3992942
TOSHIBA MG08ACA16TA	3548250
TOSHIBA MG07ACA14TA	3343724
ST16000NM001G	3098105
WDC WUH721816ALE6L4	2366036
TOSHIBA MG10ACA20TE	1665263
ST12000NM0008	1664558
ST12000NM001G	1185545
HGST HUH721212ALE604	1183774
ST8000NM0055	1176619

Name: count, dtype: int64

```
In [12]: # =====
# 11. REMOVE HIGH-CARDINALITY ID COLUMNS
# =====

print("\n" + "=" * 60)
print("HIGH-CARDINALITY ID CHECK")
print("=" * 60)

categorical_columns = df.select_dtypes(
    exclude=np.number
```

```
).columns.tolist()

id_columns = []

for column in categorical_columns:

    column_lower = column.lower()

    is_id_name = (

        column_lower in [
            "id",
            "device_id",
            "disk_id",
            "serial",
            "serial_number",
            "serialnumber"
        ]

        or column_lower.endswith(
            "_id"
        )

        or "serial" in column_lower
    )

    if is_id_name:

        unique_count = df[
            column
        ].nunique(
            dropna=False
        )

        unique_ratio = (
            unique_count
            / len(df)
        )

        print(
            f"{column}: "
            f"{unique_count:,} unique values "
            f"({unique_ratio:.2%} unique)"
        )

        if unique_ratio > 0.50:

            id_columns.append(
                column
            )

if len(id_columns) > 0:
```



```

    print(
        "\nIdentifier columns removed:"
    )

    print(
        id_columns
    )

    df = df.drop(
        columns=id_columns
    )

else:

    print(
        "\nNo high-cardinality "
        "identifier columns detected."
    )

# Refresh column lists

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()

print(
    "\nRemaining categorical columns:"
)

print(
    categorical_columns
)

gc.collect()

```

```

=====
HIGH-CARDINALITY ID CHECK
=====
serial_number: 351,095 unique values (1.15% unique)

No high-cardinality identifier columns detected.

Remaining categorical columns:
['date', 'serial_number', 'model']

```

Out[12]: 0

```

In [13]: # =====
# 12. MEMORY-SAFE CATEGORICAL ENCODING
# =====

```

```
print("\n" + "=" * 60)
print("MEMORY-SAFE CATEGORICAL ENCODING")
print("=" * 60)

# Maximum categories allowed
# for one-hot encoding

MAX_CATEGORIES = 20

columns_to_encode = []

columns_not_encoded = []

for column in categorical_columns:

    unique_count = df[
        column
    ].nunique(
        dropna=False
    )

    if unique_count <= MAX_CATEGORIES:

        columns_to_encode.append(
            column
        )

    else:

        columns_not_encoded.append(
            column
        )

print(
    "\nColumns selected for encoding:"
)

print(
    columns_to_encode
)

print(
    "\nHigh-cardinality columns NOT encoded:"
)

print(
    columns_not_encoded
)

# -----
# Convert selected columns to category
# -----
```

```
for column in columns_to_encode:

    df[column] = df[
        column
    ].astype(
        "category"
    )

# -----
# Sparse one-hot encoding
# -----

if len(columns_to_encode) > 0:

    df = pd.get_dummies(
        df,
        columns=columns_to_encode,
        drop_first=True,
        dtype="int8",
        sparse=True
    )

    print(
        "\nCategorical encoding completed."
    )

else:

    print(
        "\nNo categorical columns "
        "required encoding."
    )

print(
    "\nDataset shape after encoding:"
)

print(
    df.shape
)

show_memory_usage(
    df,
    "Memory after encoding"
)

gc.collect()
```

```
=====
MEMORY-SAFE CATEGORICAL ENCODING
=====
```

Columns selected for encoding:
[]

High-cardinality columns NOT encoded:
['date', 'serial_number', 'model']

No categorical columns required encoding.

Dataset shape after encoding:
(30597484, 19)
Memory after encoding: 9088.25 MB

Out[13]: 0

```
In [14]: # =====
# 13. CONVERT BOOLEAN COLUMNS
# =====

print("\n" + "=" * 60)
print("CONVERT BOOLEAN COLUMNS")
print("=" * 60)

boolean_columns = df.select_dtypes(
    include="bool"
).columns.tolist()

if len(boolean_columns) > 0:

    for column in boolean_columns:

        df[column] = df[
            column
        ].astype(
            "int8"
        )

    print(
        f"{len(boolean_columns)} "
        "boolean columns converted."
    )

else:

    print(
        "No boolean columns found."
    )
```

```
=====
CONVERT BOOLEAN COLUMNS
=====
No boolean columns found.
```

```
In [15]: # =====
# 14. SKEWNESS CHECK
```

```
# =====  
  
print("\n" + "=" * 60)  
print("SKEWNESS OF NUMERICAL FEATURES")  
print("=" * 60)  
  
numeric_columns = df.select_dtypes(  
    include=np.number  
) .columns.tolist()  
  
# Do not calculate skewness on target  
  
skewness_columns = (  
    numeric_columns.copy()  
)  
  
if (  
    target_column is not None  
    and target_column in skewness_columns  
):  
  
    skewness_columns.remove(  
        target_column  
    )  
  
if len(skewness_columns) > 0:  
  
    skewness = df[  
        skewness_columns  
    ].skew()  
  
    print(  
        skewness.sort_values(  
            ascending=False  
        )  
    )  
  
else:  
  
    print(  
        "No numerical features "  
        "available."  
    )
```

```
=====
SKEWNESS OF NUMERICAL FEATURES
=====
smart_9_raw          0.290257
capacity_bytes       0.055327
smart_5_normalized   0.000000
smart_5_raw          0.000000
smart_90_normalized  0.000000
smart_90_raw         0.000000
smart_187_normalized 0.000000
smart_197_normalized 0.000000
smart_187_raw        0.000000
smart_188_normalized 0.000000
smart_188_raw        0.000000
smart_198_normalized 0.000000
smart_197_raw        0.000000
smart_198_raw        0.000000
smart_9_normalized   -0.535397
dtype: float64
```

```
In [16]: # =====
# 15. LOG TRANSFORMATION
# =====

print("\n" + "=" * 60)
print("LOG TRANSFORMATION")
print("=" * 60)

transformation_columns = (
    skewness_columns.copy()
)

for column in transformation_columns:

    try:

        skew_value = df[
            column
        ].skew()

        minimum_value = df[
            column
        ].min()

        if (
            skew_value > 1
            and minimum_value >= 0
        ):

            df[column] = np.log1p(
                df[column]
            )

            print(
                f"Log transformation applied: "
```

```

        f"{column}"
    )

    except Exception as error:

        print(
            f"Skipped {column}: "
            f"{error}"
        )

print(
    "Transformation completed."
)

```

```

=====
LOG TRANSFORMATION
=====
Transformation completed.

```

```

In [17]: # =====
# 16. STANDARDIZATION
# =====

print("\n" + "=" * 60)
print("STANDARDIZATION")
print("=" * 60)

# -----
# IMPORTANT:
# Only standardize ORIGINAL CONTINUOUS NUMERICAL FEATURES.
#
# Do NOT standardize one-hot encoded columns.
# Do NOT standardize the target column.
# -----

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

# Columns created by one-hot encoding
# usually contain "__" in pandas naming,
# but we use the actual list created above
# to be completely safe.

encoded_columns = []

for column in df.columns:

    # Identify sparse dummy columns

    if hasattr(
        df[column].dtype,
        "subtype"
    )

```

```
    ):

        if (
            df[column].dtype.subtype
            == np.dtype("int8")
        ):

            encoded_columns.append(
                column
            )

# -----
# Better identification:
# exclude all columns that were created
# during categorical encoding.
# -----

encoded_columns = []

if len(columns_to_encode) > 0:

    for original_column in columns_to_encode:

        prefix = (
            str(original_column)
            + "_"
        )

        for column in df.columns:

            if str(column).startswith(
                prefix
            ):

                encoded_columns.append(
                    column
                )

# Remove duplicates

encoded_columns = list(
    dict.fromkeys(
        encoded_columns
    )
)

# -----
# Select scaling columns
# -----

scaling_columns = []

for column in numeric_columns:
```



```
# Skip target

if (
    target_column is not None
    and column == target_column
):

    continue

# Skip encoded columns

if column in encoded_columns:

    continue

# Only continuous numerical
# columns are standardized

scaling_columns.append(
    column
)

print(
    "\nNumerical columns:",
    len(numeric_columns)
)

print(
    "Encoded columns excluded:",
    len(encoded_columns)
)

print(
    "Columns selected for scaling:",
    len(scaling_columns)
)

print(
    "\nScaling columns:"
)

print(
    scaling_columns
)

# -----
# Standardize safely
# -----

if len(scaling_columns) > 0:

    print(
        "\nConverting scaling columns "
```

```
        "to float32..."
    )

    # Explicitly convert these columns
    # to floating-point BEFORE assigning
    # standardized values.

    for column in scaling_columns:

        df[column] = pd.to_numeric(
            df[column],
            errors="coerce"
        ).astype(
            np.float32
        )

    # -----
    # StandardScaler
    # -----

    scaler = StandardScaler()

    print(
        "Calculating standardized values..."
    )

    scaled_array = scaler.fit_transform(
        df[
            scaling_columns
        ]
    )

    # Convert output to float32

    scaled_array = scaled_array.astype(
        np.float32
    )

    # -----
    # SAFE COLUMN-BY-COLUMN ASSIGNMENT
    # -----

    for index, column in enumerate(
        scaling_columns
    ):

        df[column] = scaled_array[
            :, index
        ]

    # Delete temporary objects

    del scaled_array
```

```

del scaler

gc.collect()

print(
    "\nStandardization completed successfully."
)

else:

    print(
        "\nNo continuous numerical "
        "features available for scaling."
    )

```

```

=====
STANDARDIZATION
=====

```

```

Numerical columns: 16
Encoded columns excluded: 0
Columns selected for scaling: 15

```

Scaling columns:

```

['capacity_bytes', 'smart_5_normalized', 'smart_5_raw', 'smart_9_normalized', 'sm
art_9_raw', 'smart_90_normalized', 'smart_90_raw', 'smart_187_normalized', 'smart
_187_raw', 'smart_188_normalized', 'smart_188_raw', 'smart_197_normalized', 'smar
t_197_raw', 'smart_198_normalized', 'smart_198_raw']

```

```

Converting scaling columns to float32...
Calculating standardized values...

```

Standardization completed successfully.

```

In [18]: # =====
# 17. FINAL MISSING VALUE CHECK
# =====

print("\n" + "=" * 60)
print("FINAL MISSING VALUE CHECK")
print("=" * 60)

missing_final = df.isnull().sum()

total_missing = int(
    missing_final.sum()
)

print(
    "Total remaining missing values:",
    total_missing
)

```

```

if total_missing > 0:

    print(
        "\nColumns containing missing values:"
    )

    print(
        missing_final[
            missing_final > 0
        ]
    )

else:

    print(
        "No missing values remain."
    )

```

```

=====
FINAL MISSING VALUE CHECK
=====
Total remaining missing values: 0
No missing values remain.

```

```

In [19]: # =====
# 18. FINAL DUPLICATE CHECK
# =====

print("\n" + "=" * 60)
print("FINAL DUPLICATE CHECK")
print("=" * 60)

final_duplicates = (
    df.duplicated().sum()
)

print(
    "Duplicate rows:",
    final_duplicates
)

```

```

=====
FINAL DUPLICATE CHECK
=====
Duplicate rows: 0

```

```

In [20]: # =====
# 19. FINAL DATASET INFORMATION
# =====

print("\n" + "=" * 60)
print("FINAL CLEANED DATASET")
print("=" * 60)

print(
    "Original rows    :",
    f"{original_rows:}"
)

```

```
print(  
    "Original columns :",  
    original_columns  
)  
  
print(  
    "Final rows      :",  
    f"{df.shape[0]:,}"  
)  
  
print(  
    "Final columns   :",  
    df.shape[1]  
)  
  
print(  
    "\nFirst 5 rows:"  
)  
  
print(  
    df.head()  
)  
  
print(  
    "\nFinal data types:"  
)  
  
print(  
    df.dtypes  
)  
  
show_memory_usage(  
    df,  
    "Final DataFrame memory"  
)
```

=====

FINAL CLEANED DATASET

=====

Original rows : 30,943,146

Original columns : 19

Final rows : 30,597,484

Final columns : 19

First 5 rows:

	date	serial_number	model	capacity_bytes	failure	\
0	2026-01-01	000a43e7dee60010	DELLBOSS VD	-2.399328	0	
1	2026-01-01	00ac486b9bec0010	DELLBOSS VD	-2.399328	0	
2	2026-01-01	01b776cae6a0010	DELLBOSS VD	-2.399328	0	
3	2026-01-01	02308a3415be0010	DELLBOSS VD	-2.399328	0	
4	2026-01-01	0394cd7bc5d00010	DELLBOSS VD	-2.399328	0	

	smart_5_normalized	smart_5_raw	smart_9_normalized	smart_9_raw	\
0	0.0	0.0	0.224379	-0.031381	
1	0.0	0.0	0.224379	-0.031381	
2	0.0	0.0	0.224379	-0.031381	
3	0.0	0.0	0.224379	-0.031381	
4	0.0	0.0	0.224379	-0.031381	

	smart_90_normalized	smart_90_raw	smart_187_normalized	smart_187_raw	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	smart_188_normalized	smart_188_raw	smart_197_normalized	smart_197_raw	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	smart_198_normalized	smart_198_raw
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0
3	0.0	0.0
4	0.0	0.0

Final data types:

date	str
serial_number	str
model	str
capacity_bytes	float32
failure	int64
smart_5_normalized	float32
smart_5_raw	float32
smart_9_normalized	float32
smart_9_raw	float32
smart_90_normalized	float32
smart_90_raw	float32
smart_187_normalized	float32
smart_187_raw	float32
smart_188_normalized	float32
smart_188_raw	float32

```

smart_197_normalized    float32
smart_197_raw           float32
smart_198_normalized    float32
smart_198_raw           float32
dtype: object
Final DataFrame memory: 7337.45 MB

```

```

In [21]: # =====
# 20. SAVE CLEANED DATASET
# =====

print("\n" + "=" * 60)
print("SAVING CLEANED DATASET")
print("=" * 60)

output_file = os.path.join(
    OUTPUT_FOLDER,
    "cleaned_disk_smart_data.csv"
)

df.to_csv(
    output_file,
    index=False
)

print(
    "Cleaned dataset saved at:"
)

print(
    os.path.abspath(
        output_file
    )
)

```

```

=====
SAVING CLEANED DATASET
=====
Cleaned dataset saved at:
C:\Users\admin\OneDrive\Desktop\DS- Activity 1\output\cleaned_disk_smart_data.csv

```

```

In [22]: # =====
# 21. SAVE PREPROCESSING SUMMARY
# =====

print("\n" + "=" * 60)
print("SAVING PREPROCESSING SUMMARY")
print("=" * 60)

summary = {

    "Original Rows":
        original_rows,

    "Original Columns":

```

```
        original_columns,

        "Final Rows":
            df.shape[0],

        "Final Columns":
            df.shape[1],

        "Duplicates Removed":
            duplicates_before,

        "Final Duplicate Rows":
            final_duplicates,

        "Remaining Missing Values":
            total_missing,

        "ID Columns Removed":
            len(id_columns),

        "Categorical Columns Encoded":
            len(columns_to_encode),

        "Categorical Columns Not Encoded":
            len(columns_not_encoded),

        "Columns Standardized":
            len(scoring_columns)

    }

summary_df = pd.DataFrame(
    summary.items(),
    columns=[
        "Metric",
        "Value"
    ]
)

summary_file = os.path.join(
    OUTPUT_FOLDER,
    "preprocessing_summary.csv"
)

summary_df.to_csv(
    summary_file,
    index=False
)

print(
    "Preprocessing summary saved at:"
)

print(
    os.path.abspath(
```



```

        summary_file
    )
)

# =====
# FINAL MEMORY CLEANUP
# =====

gc.collect()

print("\n" + "=" * 60)
print("ALL 21 PREPROCESSING STEPS COMPLETED")
print("=" * 60)

print(
    "\nOutput files:"
)

print(
    "1. Cleaned dataset:"
)

print(
    os.path.abspath(
        output_file
    )
)

print(
    "\n2. Preprocessing summary:"
)

print(
    os.path.abspath(
        summary_file
    )
)

```

```

=====
SAVING PREPROCESSING SUMMARY
=====

```

Preprocessing summary saved at:

C:\Users\admin\OneDrive\Desktop\DS- Activity 1\output\preprocessing_summary.csv

```

=====
ALL 21 PREPROCESSING STEPS COMPLETED
=====

```

Output files:

1. Cleaned dataset:

C:\Users\admin\OneDrive\Desktop\DS- Activity 1\output\cleaned_disk_smart_data.csv

2. Preprocessing summary:

C:\Users\admin\OneDrive\Desktop\DS- Activity 1\output\preprocessing_summary.csv

In []: